Как языковая модель продолжает текст

От частот соседних слов к памяти последовательности

Следующий токен ещё не выбран

Слайд 1. Как языковая модель продолжает текст

Начнём с фразы «Студент открыл…». Как её продолжить? Можно открыть учебник, дверь, окно или личный кабинет. Если перед этим обсуждался вход на сайт, один вариант станет естественнее. Если речь шла о душной аудитории — другой. Нам нужна программа, которая учитывает уже написанное и оценивает возможные продолжения.

Слайд 2. У префикса несколько продолжений

Языковая модель задаёт распределение вероятностей следующего токена по префиксу. Внутри одного шага вероятности всех допустимых вариантов складываются в единицу. Само распределение ещё не является ответом: из него нужно выбрать токен. После выбора токен добавляется к префиксу, и следующий шаг получает уже другое начало.

На прошлой лекции вектор запроса помогал выбрать документ. Теперь выходом модели будет вектор с числом координат, равным размеру словаря: одна координата на каждый возможный следующий токен. Нейросетевой способ получить такой вектор разберём ближе к концу занятия. Сначала построим модель, у которой каждое число можно объяснить счётчиком наблюдений.

Вероятность продолжения относится к тексту и распределению модели. Она не проверяет истинность написанного. Фраза о сроке продления книги может быть привычной, но содержать неверный срок. Когда дойдём до ответов по документам, нам понадобится отдельно проверять, откуда взялось это число.

Слайд 3. У последовательности есть границы

Пока договоримся о границах. BOS обозначает начало последовательности и служит контекстом. EOS — конец, который модель должна предсказать. После EOS генерация завершается. Различие удобно увидеть на пустом тексте: слов нет, но событие завершения всё равно есть.

Вероятность текста складывается из шагов

Слайд 4. Вероятность пути — произведение

Для последовательности w_1, w_2, …, w_T цепное правило записывает вероятность как произведение условных вероятностей: $P(w_1,\ldots,w_T)=\prod_t P(w_t|w_1,\ldots,w_{t-1})$. Каждый множитель оценивает следующий токен при уже заданных предыдущих. Если мы моделируем завершённый текст, последним токеном в этой записи будет EOS.

Рассмотрим короткий путь из трёх предсказаний. Пусть нужный первый токен получает вероятность 0,5, второй при выбранном первом — 0,25, а EOS после них — 0,5. Вероятность всего пути равна $0{,}5\times0{,}25\times0{,}5=1/16$. Эти числа заданы для примера; далее получим условные вероятности из корпуса.

На каждом шаге есть и другие ветви. Внутри шага мы суммируем вероятности альтернатив, чтобы проверить нормировку. Вдоль выбранного пути перемножаем вероятности переходов. Сравним сумму вероятностей альтернатив и произведение вдоль выбранного пути. Сумма отвечает на вопрос «какие продолжения доступны сейчас», произведение — «какова вероятность именно этой последовательности».

Для длинного текста произведение быстро становится маленьким. Поэтому вычислять удобнее логарифм: логарифм произведения равен сумме логарифмов. Позже отрицательные логарифмы станут потерей, по которой сравниваются модели. Пока достаточно проследить один путь и правильно указать условие каждого множителя.

Число возможных полных историй огромно. Даже в небольшом корпусе большинство длинных префиксов встречается один раз. Если хранить отдельное распределение для каждого, новым сообщениям почти нечего будет сопоставить. Первый способ обойти это затруднение — сократить историю.

N-грамма оставляет короткий контекст

Слайд 5. Длина истории равна n − 1

Униграммная модель, n=1, предсказывает токен по общей частоте, без предыдущих слов. Биграммная, n=2, учитывает один предыдущий токен. Триграммная, n=3, — два. В число n входит и предсказываемый элемент, поэтому длина истории равна n−1.

Слайд 6. Три предложения дают счётчики

Возьмём три предложения: «студент читает книгу», «студент читает журнал», «студент возвращает книгу». Добавим к каждому BOS и EOS. После слова «студент» дважды встретилось «читает» и один раз «возвращает».

Слайд 7. Частота становится вероятностью

Биграммная оценка даёт P(читает|студент)=2/3 и P(возвращает|студент)=1/3.

После «читает» встретились «книгу» и «журнал», по одному разу. Их вероятности равны 1/2. После «возвращает» наблюдалось только «книгу». После «книгу» и «журнал» стоит EOS. Из этих счётчиков получаются переходы между словами: для каждого предыдущего слова можно перечислить продолжения, их частоты и вероятности.

Общее правило — P(w|h)=c(h,w)/c(h). Числитель считает продолжение w после истории h; знаменатель — все наблюдённые продолжения этой истории. Важно считать переходы внутри каждого предложения. Конец одного обращения не должен становиться началом истории другого.

Для триграммы первая история состоит из двух BOS. Они позволяют обработать первое слово по тому же правилу длины контекста. При этом BOS остаётся служебным входом: модель не должна генерировать его посреди ответа.

Проверим, что теряется при сокращении истории. Униграмма одинаково оценивает перестановки одних и тех же слов при том же числе EOS: множители просто меняются местами. Биграмма уже различает соседние пары. Однако более далёкие слова для неё недоступны. После «открыть» она использует одно распределение независимо от того, обсуждались раньше дверь, учебник или кабинет.

Увеличение n сохраняет больше ближайшего контекста, но делает каждую конкретную историю реже. Поэтому более длинная память не гарантирует лучших результатов на новых текстах. Это станет одним из вопросов лабораторной: три порядка будут обучены на одной выборке и оценены на одинаковых целях.

Сглаживание перераспределяет вероятность

Слайд 8. Новый переход получает ноль

В мини-корпусе переход «студент журнал» не встречался. Частотная оценка назначает ему ноль, хотя оба слова известны. Один нулевой множитель делает нулевой вероятность всего текста. Отсутствие сочетания в трёх предложениях — слабое основание считать его невозможным, поэтому изменим оценку.

Слайд 9. Добавка меняет весь знаменатель

При сглаживании add-alpha к каждому счётчику продолжения добавляется α>0. Формула принимает вид P(w|h)=(c(h,w)+α)/(c(h)+α|V|), где V — фиксированный выходной словарь. Добавка в знаменателе учитывает все |V| возможных продолжений.

Слайд 10. Сглаживание на семи токенах

Для ручного примера положим α=1. В выходном словаре семь элементов: студент, читает, возвращает, книгу, журнал, EOS, UNK. Последний обозначает неизвестное слово; BOS в этот список не входит. После «студент» наблюдалось три продолжения, поэтому новый знаменатель равен 3+7=10.

Для «читает» получаем (2+1)/10=0,3. Для «возвращает» — (1+1)/10=0,2. Каждому из пяти остальных элементов достаётся 0,1. Проверим сумму: 0,3+0,2+5×0,1=1. Раньше «читает» имело вероятность 2/3, теперь 0,3: масса для новых вариантов взята в том числе у частых наблюдённых продолжений.

Что получится для истории, которой вообще не было? Все её счётчики нулевые, и формула даст 1/7 каждому элементу. Это свойство выбранной оценки. Она пока не умеет обратиться к похожей истории или использовать смысл слов.

Слайд 11. UNK объединяет разные слова

Неизвестное сочетание нужно отличать от неизвестного слова. Для слова вне словаря нет отдельной выходной координаты. В нашей работе редкие слова обучения, встретившиеся меньше двух раз, объединяются в UNK. Затем этот же словарь применяется к validation и test. Модель оценивает получившуюся последовательность, а различия между заменёнными словами уже потеряны.

Поэтому рядом с размером словаря будем считать долю UNK. Если большая часть текста превратилась в один символ, предсказывать его может стать легче, но распознавать исходные слова модель лучше не научилась. В основной лабораторной α=0,1; add-one выше выбран для удобства ручного расчёта.

Сглаживание зависит от масштаба счётчиков. Если повторить все три предложения пять раз, несглаженная оценка «читает» останется 10/15=2/3. При add-one она станет 11/22=0,5 вместо 0,3. Новых ситуаций мы не добавили, но сделали фиксированную добавку меньше относительно наблюдений. Это ещё одна причина учитывать происхождение повторов в корпусе.

Perplexity оценивает эталонные продолжения

Слайд 12. Потеря на целевых токенах

Чтобы проверить модель, предъявим ей новые тексты и на каждой позиции возьмём вероятность правильного следующего токена. Потеря позиции равна −ln p. Если вероятность близка к единице, потеря мала; если модель считает наблюдённый токен маловероятным, потеря растёт.

Средняя потеря $H=-(1/T)\Sigma_t \ln P(w_t|\text{история})$ считается по всем целевым позициям. Perplexity равна exp(H). Меньшая perplexity соответствует меньшей средней потере

на тех же событиях. В равномерном распределении из четырёх вариантов правильный получает 1/4, H=ln4, а perplexity=4.

Слайд 13. Perplexity на трёх событиях

Вернёмся к вероятностям 0,5; 0,25; 0,5. Их произведение равно 1/16, поэтому H=ln16/3≈0,9242, а perplexity=16^(1/3)≈2,5198. В три цели включён EOS. BOS здесь задавал начало и целью не являлся.

Иногда perplexity объясняют через эффективное число вариантов. Равномерный пример помогает почувствовать масштаб, но число 2,5198 не означает, что на каждом шаге существовало столько допустимых слов. Перед нами среднее по разным распределениям и разным историям.

Слайд 14. Общие условия сравнения

Для корпуса суммируем потери всех токенов и делим на их общее число. Простое среднее perplexity документов даст другое взвешивание: короткая записка и длинное письмо получат одинаковый вес. В нашей функции вместе с метрикой будет возвращаться число целей, чтобы можно было проверить знаменатель.

Сравнение требует одинаковых текстов, токенизации, словаря и обработки границ. Словесная и посимвольная модели предсказывают разные события. При изменении порога UNK также меняется сама последовательность целей. Сначала нужно согласовать задачу, и только потом сравнивать числа.

Perplexity использует эталонный префикс. Свободная генерация будет использовать собственные выбранные токены модели. Это различие заметно даже на нашей маленькой модели: потеря на готовом тексте может быть конечной, а при продолжении собственного вывода модель может повторяться или создавать странные сочетания.

Из распределения выбирается один путь

Слайд 15. Greedy: локальный максимум

Жадный выбор, greedy, берёт самый вероятный токен на каждом шаге. Это локальное решение. Проверим на дереве из двух шагов: первая ветвь A имеет вероятность 0,6, вторая B — 0,4. Лучший следующий переход после A равен 0,4, после B — 0,9. Greedy начинает с A и получает путь вероятности 0,24. Путь через B имеет вероятность 0,36.

Чтобы это были полные распределения, у A должны быть и другие продолжения, каждое не вероятнее 0,4; например 0,35 и 0,25. У B остаток составляет 0,1. Пример показывает только локальность greedy: на первом шаге ещё не учтено, какие возможности откроются дальше.

Слайд 16. Sampling создаёт разные пути

При sampling следующий токен выбирается случайно с вероятностями модели. Самый частый вариант встречается чаще, но остальные тоже могут быть выбраны. Один и тот же префикс поэтому способен дать несколько продолжений. Для повторения опыта используем фиксированный seed; для наблюдения вариативности — несколько заранее выбранных значений.

Это видно в продолжениях биграммы из нашей работы. После общего префикса «как» при seed 7 появились «подать социальной стипендии», при seed 42 — начало «получить другую сообщить…», при seed 99 — «нужно ли <UNK>…». Два последних фрагмента здесь сокращены; полные строки сохраняются в ноутбуке. На первом шаге у опытов одинаковое распределение, но после разных выбранных токенов истории расходятся и следующие распределения могут различаться.

После выбора токен добавляется к истории. Если выбран EOS, останавливаемся. Если EOS долго не появляется, нужен предел числа новых предсказаний. В лабораторной он равен 40. Причина остановки будет возвращаться вместе с текстом, потому что достижение предела и предсказанное завершение — разные события.

UNK оставим видимым. Иначе строка может выглядеть гладкой после удаления как раз тех выходов, которые показывают ограничение словаря. Прочитаем несколько результатов и попробуем объяснить их через вероятности переходов. На седьмой лекции появятся temperature и top-p, но основное разделение останется прежним: модель даёт распределение, правило генерации выбирает из него путь.

RNN переносит информацию в состоянии

Слайд 17. RNN обновляет состояние

N-граммная модель хранит конкретный короткий хвост. Рекуррентная нейросеть вместо этого обновляет вектор состояния после каждого токена. Предыдущее состояние и текущий вход вместе определяют новое. Его можно рассматривать как сжатое представление уже прочитанного префикса.

Пусть e_t — эмбеддинг текущего токена. Простое обновление имеет вид $h_t=\tanh(W_x e_t+W_h h_{t-1}+b_h)$. Матрицы преобразуют вход и прежнее состояние, а tanh добавляет нелинейность. На всех позициях используются одни и те же параметры. Это одна сеть, развёрнутая по шагам последовательности.

Слайд 18. Нулевой вход не стирает прошлое

Возьмём скалярный пример $h_t=\tanh(0{,}5h_{t-1}+e_t)$, $h_0=0$. При $e_1=1$ получим $h_1\approx0{,}7616$. Если $e_2=0$, то $h_2\approx0{,}3634$. Второй вход нулевой, но влияние предыдущего ещё

сохраняется. Для двух нулевых входов оба состояния были бы нулевыми. Эти параметры заданы вручную, чтобы увидеть действие связи.

Для языковой модели состояние преобразуется в оценки словаря: $z_t=W_o h_t+b_o$. Затем softmax даёт вероятности следующего токена. Если состояние уже обработало w_t, его выход относится к w_{t+1}. Важно сохранить этот сдвиг на одну позицию, чтобы не подать целевое слово на вход до его предсказания.

Вектор фиксированной длины не хранит точную копию любого прошлого текста. При повторных преобразованиях сведения могут смешиваться и теряться. Но общие обучаемые параметры позволяют использовать закономерности сразу для многих историй, вместо отдельной частоты каждого полного сочетания.

Обучение подаёт правильный префикс

Слайд 19. Входы и цели сдвинуты

Для «студент читает книгу» запишем два ряда. Входы: BOS, студент, читает, книгу. Цели: студент, читает, книгу, EOS. Сдвиг равен одной позиции. На каждом шаге считаем −ln вероятности соответствующей цели, затем объединяем потери.

Слайд 20. Откуда берётся следующий вход

В режиме teacher forcing следующий вход берётся из эталонного текста. Если целевое слово — «книгу», а модель предпочла «журнал», при обучении следующий шаг всё равно получит «книгу». При свободной генерации дальше пойдёт выбранный самой моделью токен. Ошибка тогда меняет все последующие условия.

Чтобы уменьшить потерю, нужно изменить веса эмбеддингов, обновления состояния и выходного слоя. Обратное распространение проходит по развёрнутой цепочке вычислений. Поскольку одни и те же веса используются на многих шагах, вклады в их градиент суммируются по разным позициям. Как в классификаторе второй лекции, параметры изменяются по тому, как они влияют на ошибку; теперь к этому добавилась зависимость между шагами.

Слайд 21. LSTM управляет сохранением

На длинной цепочке градиент может становиться очень маленьким или большим. Это затрудняет обучение дальних зависимостей. LSTM и GRU меняют устройство рекуррентного блока, чтобы гибче управлять сохранением информации. Например, LSTM поддерживает отдельное состояние ячейки и обучаемые коэффициенты сохранения, добавления и выдачи информации.

Эти коэффициенты обычно непрерывные, а не жёсткие переключатели. На схеме полезнее проследить, какая часть прежнего состояния сохраняется и какая

добавляется, чем запоминать все названия ворот сразу. Такие блоки улучшают пути передачи информации, но состояние по-прежнему ограничено.

При пакетном обучении короткие последовательности дополняются padding. Заполнители не должны становиться целями языковой оценки. Иначе множество легко предсказываемых технических позиций изменит потерю. Это знакомый вопрос о знаменателе: какие события мы фактически усреднили?

Что покажет сравнение трёх порядков

Слайд 22. Валидация выбирает биграмму

В работе возьмём по одному обращению из каждой группы: 72 текста для обучения, 24 для валидации и 24 для теста. Общий словарь строится на train, min_count=2. Сравниваются n=1,2,3 при α=0,1. После выбора по validation оценивается выбранный порядок на test.

В выполненном опыте словарь содержит 47 выходных элементов. На валидации perplexity униграммы равна примерно 6,6388, биграммы — 4,9800, триграммы — 6,0382. Поэтому выбирается n=2. Триграмма имеет меньшую обучающую потерю, но это преимущество не сохранилось на новых историях.

Слайд 23. Perplexity и покрытие словаря

Тестовая perplexity биграммы составляет около 4,9404 на 190 целях. Чтобы понять абсолютную величину, посмотрим на покрытие: UNK заменил примерно 58,02% слов train, 68,48% validation и 71,08% test. Доля UNK считается по обычным словам, без EOS; в число целей perplexity EOS входит. Большая доля исходных различий потеряна. Более низкую тестовую perplexity по сравнению с обучающей нельзя объяснять только хорошим переносом: состав последовательностей после замены тоже различается.

Слайд 24. Прочитаем продолжения

Генерация делает ограничение заметным. Greedy после «как» повторил UNK 40 раз и достиг предела. Sampling с seed 7 дал «как подать социальной стипендии» и EOS на четвёртом новом предсказании. Тема узнаваема, но фраза построена с грамматической ошибкой. Другие seed дали более длинные смеси конструкций. Чтение этих выходов помогает понять то, чего не видно в одной метрике.

От цепочки состояний к attention

Вернёмся к начальной фразе «Студент открыл…». Теперь мы можем назвать несколько отдельных решений: какие токены доступны в словаре, какой контекст

видит модель, как получены вероятности и как выбран следующий токен. Если результат плох, эти вопросы помогают найти место для изменения.

На защите лабораторной разберём два коротких случая. Первый — невстречавшаяся история при add-alpha: почему сумма вероятностей всё равно равна единице? Второй — одинаковая perplexity при разных токенизаторах: что именно сравнивается и каких сведений не хватает для вывода? Ответ лучше начинать с формулы и конкретных событий.

У RNN новое состояние зависит от предыдущего, и сведения проходят по цепочке. Следующая лекция предлагает другой механизм: позиция сможет напрямую сравнить свой запрос с доступными позициями текста и смешать их значения. Для этого нам пригодятся уже знакомые векторы, softmax и правило, которое не позволяет увидеть будущее при предсказании следующего токена.

Для дальнейшего чтения

– Jurafsky, Martin. Speech and Language Processing — n-граммные модели и оценка вероятностей.

– Dive into Deep Learning: Language Models — последовательности и языковое моделирование.

– Dive into Deep Learning: Recurrent Neural Networks — состояние, сдвиг входов и целей, perplexity.

Мини-корпус и скалярное обновление состояния — учебные примеры. Числа основной работы получены на авторском корпусе обращений «Север».